

Project Workflow: Week 6

Phase: Core Logic – Collision & State

1. Objective

The goal of Week 6 was to give the ball meaningful interaction with the rest of the game world and to formalize how the game moves between different states. This included:

1. Implementing obstacle collision detection logic.
2. Handling ball-paddle and ball-obstacle interactions.
3. Managing overall game state (running, paused, game over).

2. Detailed Task Breakdown

- Implementing Obstacle Collision Detection

Built the core collision-checking logic that determines whether the ball has come into contact with an obstacle on the playing field. This uses bounding-box (or circle-vs-rectangle, depending on obstacle shape) overlap checks performed every frame against each obstacle's current position, so contact is detected as soon as the ball's edge crosses into an obstacle's boundary rather than only after a full overlap.

- Handling Ball-Paddle and Ball-Obstacle Interactions

Extended the collision system to react differently depending on what the ball hits. On paddle contact, the ball's vertical (and where relevant, horizontal) velocity is reflected based on where it struck the paddle, giving the player some control over the rebound angle. On obstacle contact, the ball bounces off using a simple reflection of its velocity component along the axis of impact, and the obstacle can be marked as hit/destroyed or left persistent depending on its type. Care was taken to prevent the ball from registering multiple collisions with the same obstacle in consecutive frames while it is still overlapping.

- Managing Overall Game State

Introduced a simple state system (e.g., an enum or set of boolean/int flags) to track whether the game is RUNNING, PAUSED, or GAME_OVER. The main update loop checks the current state before processing physics and input: updates only run while RUNNING, input is limited to a resume/pause toggle while PAUSED, and only a restart action is accepted while GAME_OVER. This keeps the collision and movement logic in Week 6 cleanly separated from whatever screen/UI is currently being shown to the player.

3. Deliverables

1. Collision Detection: Reusable overlap-checking logic between the ball and obstacles.
2. Interaction Handling: Distinct, correctly-reflected bounce behavior for ball-paddle and ball-obstacle contact.
3. Game State Manager: A tracked state (Running / Paused / Game Over) that gates which logic runs each frame.
4. Updated Game Loop: Collision checks and state checks wired into the existing update/render cycle.

4. Tools & Resources

❖ Development Environment

1. Java JDK 8+
2. IntelliJ IDEA / VS Code

❖ Reference Materials

Java 2D collision detection concepts (bounding box / circle-rectangle overlap), vector reflection for bounce physics, and state-machine design patterns for managing game states (Running/Paused/Game Over).